

华东师范大学数据科学与工程学院实验报告

课程名称: AI 基础	年级: 2024	上机实践时间: 2026.03.29
指导教师: 杨彬	姓名: 孙育泉	
上机实践名称: Maze	学号: 10234900421	

I 实验任务

给定一个 $n \times m$ 的二维整数数组，用来表示一个迷宫，数组中只包含 0 或 1，其中 0 表示可以走的路，1 表示不可通过的墙壁。最初，有个人位于左上角 $(1, 1)$ 处，已知该人每次可以向上、下、左、右任意一个方向移动一个位置。

请问，该人从左上角移动至右下角 (n, m) 处最少需要移动多少次。数据保证 $(1, 1)$ 与 (n, m) 处的值均为 0，且至少存在一条路径可以到达。并且需要进行可视化。

考虑使用这些算法：DFS, BFS, Dijkstra, A*。

II 使用环境

编程语言：Python 3

III 实验过程

 **Idea**

为了代码的可维护性与解藕性，这里对代码进行了很大的改动，进行了模块化。

首先，为了进行后续的可视化与维护每次探索的状态，我们将迷宫分装成了一个 Maze 类。

py

```
1 # maze.py
2 from dataclasses import dataclass
3 from typing import List, Tuple, Set
4
5
6 @dataclass
7 class SolverState:
8     """用于寻路算法与可视化解耦的数据模型，保存每一步的快照"""
9
10     current: Tuple[int, int]
11     explored: List[Tuple[int, int]]
12     frontier: List[Tuple[int, int]] # 待探索队列（栈/队列/优先队列）
13     path: List[Tuple[int, int]]
14
15
16 class Maze:
17     """迷宫的基础数据结构"""
18
19     def __init__(
20         self,
21         grid: List[List[int]],
22         start: Tuple[int, int] = (0, 0),
```

```

23     goal: Tuple[int, int] = None,
24 ):
25     self.grid = grid
26     self.rows = len(grid)
27     self.cols = len(grid[0])
28     self.start = start
29     self.goal = goal if goal else (self.rows - 1, self.cols - 1)
30
31     def is_wall(self, r: int, c: int) → bool:
32         return self.grid[r][c] == 1
33
34     def is_valid_move(self, r: int, c: int) → bool:
35         return 0 ≤ r < self.rows and 0 ≤ c < self.cols and not
            self.is_wall(r, c)
36
37     def get_neighbors(self, r: int, c: int) → List[Tuple[int, int]]:
38         directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
39         neighbors = []
40         for dr, dc in directions:
41             nr, nc = r + dr, c + dc
42             if self.is_valid_move(nr, nc):
43                 neighbors.append((nr, nc))
44         return neighbors
45

```

然后，我们需要生成迷宫，用于探究算法在不同的迷宫类型下的性能表现。

py

```

1 # generators.py
2 import random
3 from collections import deque
4 from maze import Maze
5
6
7 class MazeGenerator:
8     """
9     迷宫生成器：包含多种不同拓扑结构的迷宫生成策略。
10    所有生成方法均返回 maze.Maze 类的实例。
11    """
12
13    @staticmethod
14    def _is_solvable(grid: list[list[int]]) → bool:
15        """内部辅助函数：使用轻量级 BFS 快速验证网格是否至少存在一条解"""
16        ...
17
18    @staticmethod
19    def generate_random_density(
20        rows: int, cols: int, wall_probability: float = 0.25
21    ) → Maze:
22        """生成随机密度的迷宫（保证有解）"""
23        ...
24
25    @staticmethod
26    def generate_perfect_maze(rows: int, cols: int) → Maze:
27        """使用 Randomized DFS 生成完美迷宫（高曲折度，长且深的死胡同）"""
28        ...
29
30    @staticmethod

```

```
31     def generate_braid_maze(  
32         rows: int, cols: int, braid_probability: float = 0.5  
33     ) → Maze:  
34         """生成编织迷宫（多解迷宫，存在环路）"""  
35         ...  
36  
37     @staticmethod  
38     def generate_cave_maze(  
39         rows: int, cols: int, initial_wall_prob: float = 0.45, iterations: int  
40         = 4  
41     ) → Maze:  
42         """利用细胞自动机生成类似自然洞穴的迷宫"""  
43         ...  
44  
45     @staticmethod  
46     def generate_recursive_division(rows: int, cols: int) → Maze:  
47         """使用递归分割法生成迷宫（拥有长直走廊和整齐的矩形区块）"""  
48         ...  
49  
50     @staticmethod  
51     def generate_prim_maze(rows: int, cols: int) → Maze:  
52         """使用随机 Prim 算法生成迷宫（极多超短死胡同，强烈的中心辐射感）"""  
53         ...
```

IV 总结

i Info

完整代码见压缩包其余代码文件.